

Dokumentation des Billard Spiels

WASP Projekt

Studiengang: Allgemeine Informatik

Autoren:

Dimitrij Pivovar, Ivan Kurilin

Betreut von:

Prof. Dr. Wolfgang Konen

Dr. rer. nat., Dipl.-Phys.

Köln, 31. März 2025

Inhaltsverzeichnis

1	Einleitung	1
2	Basis	1
2.1	Spielregeln	1
2.2	Punktesystem	1
2.3	Spielumgebung	1
2.4	Menü	2
2.5	Spielsteuerung	3
2.6	Physikalische Interaktionen	3
3	Implementierung	4
3.1	Physik	4
3.1.1	Kollision zwischen zwei Kugeln	4
3.1.2	Kollision mit Banden	5
3.1.3	Kollision mit Pockets	6
3.1.4	Kugelanimation beim Einlochen	6
3.1.5	Rotation	8
3.1.6	Reibung	8
3.2	Klassen	9
3.2.1	UserInterface	9
3.2.2	MyCanvas	9
3.2.3	Ball	9
3.2.4	BallType Enum	10
3.2.5	Billard_ssds (Main Datei)	10
3.2.6	BillardCue	10
3.2.7	GameController	11
3.2.8	GameState Enum	11
3.2.9	Player	11
3.2.10	Pockets	12
3.2.11	ShootingBar	12
3.2.12	Table	13
3.2.13	VectorDrawer	13
3.2.14	SoundController	14
4	Texturierung	15
5	Artefakte	16
5.1	3D-Effekt	16
5.2	Pockets	16
5.3	Rotation	16
5.4	Vektoranzeige	17
6	Vergleich mit anderen Arbeiten	17
6.1	Studienarbeit aus Koblenz	17
6.2	Online Billardspiele	17
7	Fazit	18

1 Einleitung

In diesem Projekt haben wir eine Billard Simulation entwickelt, die sowohl eine realistische grafische Darstellung als auch physikalische Interaktionen der Kugeln berücksichtigt. Ziel war es, eine funktionierende Simulation zu erstellen, die die Bewegungen und Kollisionen der Kugeln realistisch darstellt. Das Projekt stellt eine interessante Herausforderung im Bereich der Spieleentwicklung dar, da es sowohl unterhaltsame als auch physikalische Simulationen kombiniert. Es bietet wertvolle Einblicke in die Entwicklung realistischer Spiele und die Anwendung von Kollisionserkennung und -berechnungen, die auch in anderen Bereichen der Spieleentwicklung relevant sind.

Diese Dokumentation bietet einen Überblick darüber, wie das Programm funktioniert, wie wir das System evaluiert haben, sowie eine Beschreibung der auftretenden Artefakte und Herausforderungen.

2 Basis

2.1 Spielregeln

Das Spiel basiert auf der 8-Ball Variante des Billards. Zwei Spieler treten gegeneinander an. Zu Beginn wird eine der beiden Gruppen von Kugeln zugewiesen: entweder die SOLID oder die STRIPE Kugeln. Ziel des Spiels ist es, alle Kugeln der eigenen Gruppe korrekt in die Taschen zu versenken und zuletzt die schwarze 8-Ball korrekt zu versenken, um zu gewinnen.

Der Spieler, der seine Kugeln zuerst versenkt, muss danach die schwarze 8-Ball versenken, um das Spiel zu gewinnen. Versenkt der Spieler die 8-Ball vorzeitig oder auf unkorrekte Weise, verliert er das Spiel.

2.2 Punktesystem

Das Punktesystem basiert auf den versenkten Kugeln. Jeder versenkte Ball der eigenen Gruppe bringt einen Punkt. Die schwarze Kugel bringt zusätzlich 8 Punkte, wenn sie korrekt versenkt wird. Das Spiel endet, wenn ein Spieler alle Kugeln seiner Gruppe versenkt, und die 8-Ball korrekt eingelocht hat. Der Spieler mit den meisten Punkten gewinnt das Spiel.

Die Punkte könnten z.B. bei mehreren Spielen hintereinander verwendet werden, um die Ergebnisse zu summieren und nach mehreren Spielen einen Gesamtsieger zu bestimmen.

2.3 Spielumgebung

Die Spielumgebung besteht aus einem rechteckigen Billardtisch, der die Basis des Spiels bildet. Der Tisch ist mit einer Textur versehen, die die Oberfläche des Billardtisches darstellt, sowie mit Banden an den Rändern, die für die Kollisionen der Kugeln wichtig sind.

Es gibt insgesamt 16 Kugeln: eine weiße Spielkugel und 15 nummerierte Kugeln, die in zwei Gruppen unterteilt sind – volle und halbe Kugeln. Der Tisch ist mit sechs Taschen ausgestattet, in denen die Kugeln versenkt werden können. Diese Taschen befinden sich in den vier Ecken des Tisches und an den beiden langen Seiten.

Die Kugeln bewegen sich auf der Oberfläche des Tisches und interagieren mit den Banden sowie mit anderen Kugeln. Die Physikengine berechnet die Kollisionen und die Bewegungen der Kugeln, um eine realistische Darstellung des Spiels zu gewährleisten.

2.4 Menü

Beim Spielstart, bevor die eigentliche Billardrunde beginnt, wird ein Menü angezeigt, das verschiedene physikalische Parameter anpassen lässt. Dazu gehören die Reibung (Friction), der Kugelradius (Ball Radius) und die Kugelmasse (Ball Masse).

Die Reibung bestimmt den Widerstand der Bälle auf der Spielfläche und beeinflusst ihr Abbremsverhalten. Ein höherer Reibungswert führt dazu, dass die Bälle schneller zum Stillstand kommen, wodurch das natürliche Verhalten der Kugeln realistisch nachgebildet wird.

Die Einstellung des Kugelradius beeinflusst sowohl die Größe der Bälle als auch die Dimensionen der Taschen, sodass die Skalierung einheitlich bleibt. Dadurch bleibt der Schwierigkeitsgrad beim Versenken der Kugeln konstant.

Die Kugelmasse verändert das Verhalten beim Stoß. Eine höhere Masse führt zu einer geringeren Endgeschwindigkeit der Kugel nach dem Stoß, was die Dynamik des Spiels beeinflusst.

Das Menü kann jederzeit auch während des Spiels aufgerufen werden (oben links). Es dient somit auch als Pausenfunktion oder ermöglicht das Beenden des Spiels.



Abbildung 1: Menü

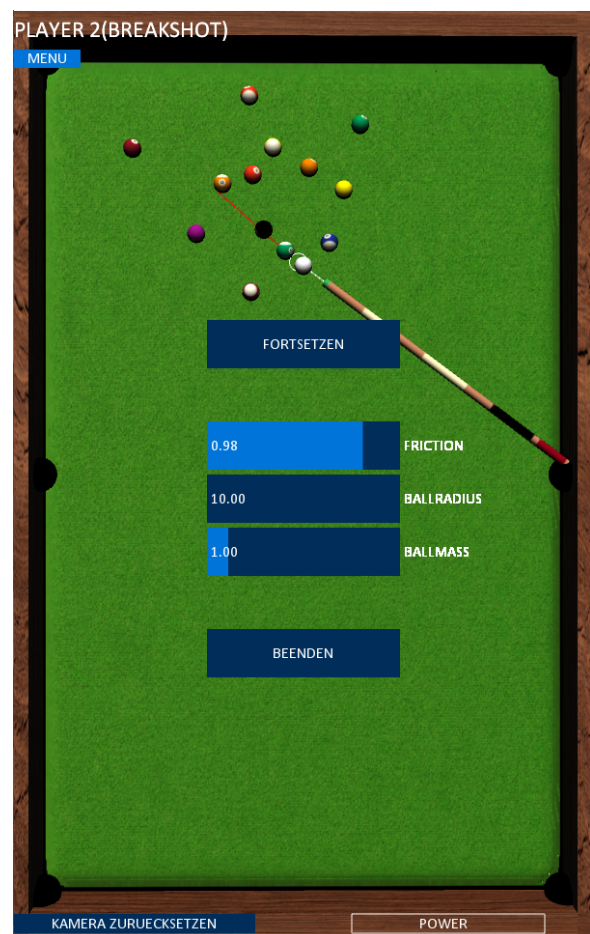


Abbildung 2: Menü während des Spiels

2.5 Spielsteuerung

Die Steuerung des Spiels erfolgt vollständig mit der Maus.

Zu Beginn des Spiels muss zunächst die weiße Kugel platziert werden.

Anschließend hat der Spieler die Möglichkeit, den Queue auf die Bälle zu zielen. Sobald das Ziel ausgerichtet ist, wird mit der linken Maustaste geklickt und die Maus entlang der Y-Achse bewegt, um die Stärke des Schusses zu bestimmen. Die aktuelle Schussstärke wird in der Shooting Bar Leiste am unteren Bildschirmrand, angezeigt.

Sobald die gewünschte Stärke erreicht ist, lässt man die Maustaste los, und der Schuss wird ausgeführt. Dieser Vorgang wird wiederholt, bis das Spiel vorbei ist.

Im Fall eines Fouls (wenn die weiße Kugel in eine Tasche fällt) darf der nächste Spieler die Kugel an einer beliebigen Stelle auf dem Tisch platzieren.



Abbildung 3: Spielstart (old)



Abbildung 4: Spielstart (new)

2.6 Physikalische Interaktionen

Die physikalischen Interaktionen im Spiel werden durch eine einfache Physikengine realisiert, die auf Processing basiert. Diese Engine berechnet die Bewegungen und Kollisionen der Kugeln sowie deren Interaktionen mit den Banden und dem Tisch. Processing ist eine Java basierte Entwicklungsumgebung, die es ermöglicht, grafische Elemente und physikalische Berechnungen einfach zu integrieren.

Die Kugeln bewegen sich entsprechend ihrer Geschwindigkeit und Richtung, wobei die Engine die Auswirkungen von Kräften wie Beschleunigung und Reibung berücksichtigt. Kollisionen zwischen Kugeln werden unter Berücksichtigung von Impuls und Energieerhaltung berechnet. Bei einer Kollision ändern die Kugeln ihre Richtung und Geschwindigkeit je nach dem Winkel und der Geschwindigkeit der Kollision.

Mit jeder Ausführung der `draw()` Methode in Processing wird die aktuelle Szene gerendert, wobei die Positionsänderungen und Kollisionen der Kugeln kontinuierlich aktualisiert und dargestellt werden. Diese wiederholte Darstellung sorgt für eine flüssige und dynamische Simulation der physikalischen Interaktionen und ermöglicht eine realistische Darstellung des Spiels.

3 Implementierung

In dieser Sektion wird die Implementierung des Billardspiels beschrieben. Die Physik, verschiedene Klassen und ihre Funktionen werden hier erläutert, um zu verdeutlichen, wie das System aufgebaut ist und wie die einzelnen Komponenten zusammenarbeiten.

3.1 Physik

3.1.1 Kollision zwischen zwei Kugeln

Die Kollision zwischen zwei Kugeln ist ein zentrales Element des Spiels. Um eine Kollision auf der Spielfläche zu erkennen, werden die Koordinaten der beiden beteiligten Bälle benötigt. Die Bälle werden mithilfe eines Bruteforce Algorithmus paarweise überprüft, indem alle Bälle in der Liste durchlaufen werden, um festzustellen, ob eine Kollision stattfindet.

Eine Kollision tritt auf, wenn der Abstand zwischen den Mittelpunkten der beiden Kugeln kleiner oder gleich der Summe ihrer Radien ist.

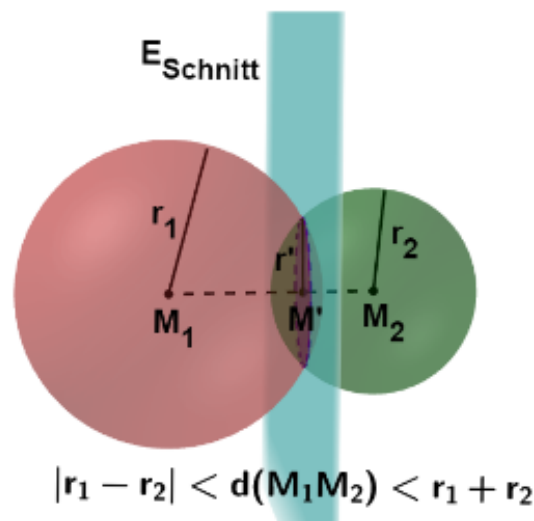


Abbildung 5: Kollision der zwei Kugeln [2]

Beispiel im Code:

```

class CBalls {..
void bruteforce() {
    for (Ball b1 : ballContainer) {
        for (Ball b2 : ballContainer) {
            if (b1 != b2) {
                float dx = (float)(b1.sx - b2.sx);
                float dy = (float)(b1.sy - b2.sy);
                float distance = (float)Math.sqrt(dx * dx + dy * dy);
                if (distance <= b1.Radius() + b2.Radius()) {
                    collisionanswer(distance, b1, b2, dx, dy);
                }
            }
        }
    }
}
..}

```

In der Kollisionsantwort wird der elastische Stoß simuliert. Dies geschieht, indem der neue Einheitsvektor berechnet wird, der die Bewegungsrichtung entlang der zurückgelegten Distanz projiziert. Anschließend wird diese Projektion auf die Geschwindigkeiten der beiden Bälle übertragen.

Dabei gilt der Energieerhaltungssatz, der besagt, dass die Gesamtenergie vor und nach dem Stoß identisch bleibt. [1] Dies lässt sich mit folgender Formel ausdrücken:

$$\frac{1}{2}m_1v_1^2 + \frac{1}{2}m_2v_2^2 = \frac{1}{2}m_1v_1'^2 + \frac{1}{2}m_2v_2'^2 \quad (1)$$

3.1.2 Kollision mit Banden

Für die Kollision mit den Banden werden vier Eigenschaften des Tisches benötigt: ‘left-wall_x’, ‘rightwall_x’, ‘floor_y’ und ‘ceiling_y’. Diese Werte werden zu Beginn festgelegt, für die Instanziierung des Tisches verwendet und können global genutzt werden.

Eine Kollision tritt auf, wenn die Ballkoordinaten in Kombination mit der vertikalen Verschiebung dy eine bestimmte Grenze überschreiten. Hierbei beschreibt dy die Differenz der aktuellen Position zur nächsten berechneten Position in der nächsten Zeiteinheit.

Nach einer Kollision wird die Geschwindigkeit der Kugel um den Faktor 0.9 reduziert, und die Bewegungsrichtung reflektiert, indem das Vorzeichen der Geschwindigkeit umgekehrt wird.

Ein Beispiel für eine Kollision mit der unteren Bande (‘floor_y’):

```

class Ball {..
if ((this.sy + dy > floor_y - this.radius)) {
    dy = floor_y - this.radius - this.sy;
    this.vy = -0.9 * this.vy;
}
..}

```

3.1.3 Kollision mit Pockets

Die Erkennung einer Kollision mit den Pockets erfolgt nach einem ähnlichen Prinzip wie die Kollision zwischen zwei Kugeln. Der wesentliche Unterschied liegt in der Kollisionsbedingung:

$$\text{distance} < b.\text{Radius}() + \frac{\text{table.pockets.pocketRadius}}{2}$$

anstelle von

$$\text{distance} \leq b1.\text{Radius}() + b2.\text{Radius}()$$

Ist die berechnete Distanz kleiner als der angegebene Schwellenwert, wird die Kugel als eingelocht betrachtet. Anschließend bestimmt der Algorithmus, abhängig von der Art der eingelochten Kugel, welche Auswirkungen dies auf den Spielstand hat. Dies umfasst unter anderem die Vergabe von Punkten, den möglichen Wechsel des Spielers sowie die Bewertung eines möglichen Foulspiels.

3.1.4 Kugelanimation beim Einlochen

Dieser Schritt existiert in der realen Welt nicht, stellt jedoch in unserer Simulation einen wichtigen Bestandteil des Spielerlebnisses dar.

Um das Einlochen visuell ansprechender zu gestalten, wurde eine Animation implementiert, die den Vorgang realistischer erscheinen lässt. Dabei wird eine eingelochte Kugel schrittweise verkleinert, bis ihr Skalierungsfaktor einen Schwellenwert von 0,5 erreicht hat. Sobald dieser Wert unterschritten wird, wird die Kugel nicht mehr angezeigt.

Zusätzlich wird durch die Variable `isFadingOut` sichergestellt, dass für diese Kugel keine Kollisionen mehr überprüft werden. In der darauffolgenden Iteration wird die Kugel endgültig aus der Liste der aktiven Bälle entfernt.



Abbildung 6: Kugelanimation (old)

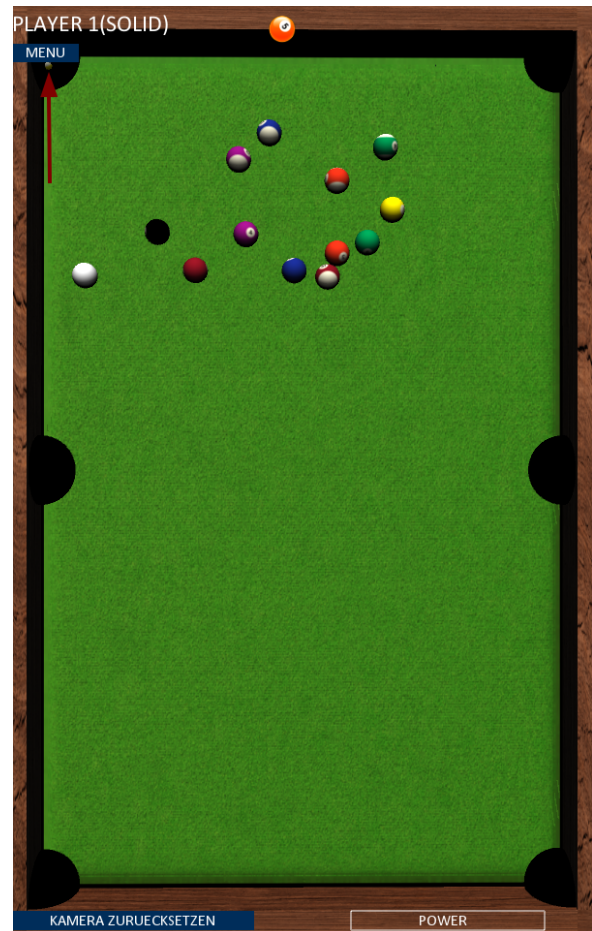


Abbildung 7: Kugelanimation (new)

Die Animation erfolgt über die folgende Methode:

```
class Ball {
void animateRemoval() {
    if (this.pocketed && scale > 0.5) {
        scale *= 0.99;
        our_sphere.scale(0.99);
    }
    if (this.pocketed && scale <= 0.5) {
        isFadingOut = true;
        setVisibility(false);
    }
}
..}
```

Im Fall einer weißen Kugel, muss sie nach Abschluss der Animation erneut auf dem Tisch platziert werden. Dies erfolgt in mehreren Schritten:

```

class CBalls {..
void placeWhiteBallAfterFoul() {
    Ball ball = getWhiteBall();
    if (ball.scale <= 0.5){
        ball.pocketed = false;
        ball.isFadingOut = false;
        ball.our_sphere.scale(2);
        ball.scale = 1;
    }
    if ( ball.scale == 1){
        ball.setVisibility(true);
        ball.sx = mouseX;
        ball.sy = mouseY;
    }
}
..}

```

Durch diese Vorgehensweise wird das Einlochen optisch ansprechend gestaltet, ohne dabei die Spielmechanik zu beeinflussen.

3.1.5 Rotation

Die Rotation eines Objekts in einem Raum kann durch das Kreuzprodukt zwischen der Richtung des Objekts (Vektor (V_x, V_y, V_z)) und einer proportionalen Richtung (Vektor $(-V_y, V_x, V_z)$) beschrieben werden.

Die Rotation des Objekts wird durch die Änderung der Winkel `angleX` und `angleY` basierend auf den Geschwindigkeitskomponenten v_x und v_y , dem Radius und dem Zeitschritt DT berechnet.

```

class Ball {..
this.angleX -= (this.vy / this.radius) * DT;
this.angleY += (this.vx / this.radius) * DT;
..}

```

Die größte Herausforderung bei der Implementierung der Rotation besteht darin, das Objekt korrekt zu drehen, wenn es nicht zentriert zur Kamera ausgerichtet ist.

Falls das Bild vollständig umgedreht ist, kann dies abgefangen werden, indem der negative Cosinus der Drehung auf der X-Achse verwendet wird. Dabei wird die Y-Achse in die entgegengesetzte Richtung gedreht:

```

class Ball {..
if (cos((float) this.angleX) < 0) {
    this.angleY -= (this.vx / this.radius) * DT * 2;
}
..}

```

3.1.6 Reibung

Um ein nahezu realistisches Verhalten zu simulieren, wird der Ball durch eine Reibungskraft (`FRICTION`) verlangsamt. Diese Reibung reduziert die Geschwindigkeit des Balls bei

jedem Frame leicht, während er rollt.

```
class Ball {..  
  this.vx *= FRICTION;  
  this.vy *= FRICTION;  
  ..}
```

3.2 Klassen

3.2.1 UserInterface

Die Klasse UserInterface (Abk. UI) ist für die Darstellung der gesamten Benutzeroberfläche verantwortlich. Dazu gehören das Hauptmenü, MyCanvas sowie die Möglichkeit, die Kamera zurückzusetzen.

Für eine verbesserte Interaktion werden die Menüpunkte hervorgehoben, sobald der Mauszeiger darüberfährt. Dies dient als visuelles Feedback, um anzuzeigen, dass der Punkt anklickbar ist.

3.2.2 MyCanvas

MyCanvas zeigt Informationen über den Kugeltyp sowie die Anzahl der bereits eingelochten Kugeln des aktuell spielenden Spielers an. Zusätzlich visualisiert es die versenkten Kugeln, was die Übersichtlichkeit verbessert und zur benutzerfreundlichen Erfassung des Spielverlaufs beiträgt.

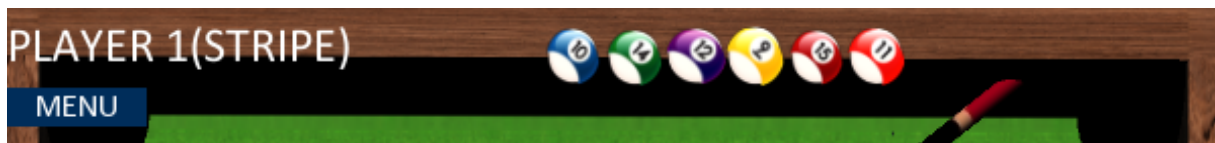


Abbildung 8: MyCanvas

3.2.3 Ball

Die Ball Klasse wird durch die PShape Klasse repräsentiert. Dies hat den Vorteil, dass wir eine Sphere erstellen können und mithilfe der Methode `setTexture(PImage)` eine Textur auf den Ball projizieren können.

Die Ball Klasse ist für die Initialisierung der Kugelposition verantwortlich. Sie setzt die Startpositionen der Kugeln, die in einem Dreieck angeordnet sind, was die typische Startposition im Billardspiel darstellt. Diese Anordnung erfolgt durch die Methode `initializePosition()`.

Darüber hinaus verwaltet die Klasse die physikalischen Eigenschaften des Balls. Sie berechnet die Rotation, die Bewegung im Raum sowie die Kollisionen mit den Banden. Die Klasse verteilt außerdem die Reibung auf die Geschwindigkeit des Balls, um realistische Bewegungen zu gewährleisten. Wenn der Ball in eine Tasche fällt, wird er mithilfe einer Animationsmethode animiert, die seine Bewegung und das Verschwinden des Balls aus dem Spiel simuliert.

Die Ball Klasse ist somit für alle grundlegenden physikalischen und visuellen Interaktionen des Balls im Spiel verantwortlich.

3.2.4 BallType Enum

Die BallType Enum wird verwendet, um die verschiedenen Ballarten im Billardspiel zu unterscheiden. Diese Enum enthält die folgenden Werte:

- NONE – Ein Wert, der für keinen Ball verwendet wird.
- SOLID – Für die Ballsorte mit einfarbigen Kugeln.
- STRIPE – Für die Ballsorte mit gestreiften Kugeln.
- BLACK – Die schwarze Kugel, die für den letzten Stoß benötigt wird.
- WHITE – Die weiße Kugel, die für den Stoß verwendet wird.

Diese Enum ist entscheidend für die Implementierung der Spielregeln, da sie es den Spielern ermöglicht, zwischen den verschiedenen Balltypen zu unterscheiden und die Regeln entsprechend anzuwenden.

3.2.5 Billard_ssds (Main Datei)

Die Datei Billard_ssds stellt den Einstiegspunkt des Spiels dar und übernimmt die zentrale Steuerung. In der Setup Methode wird zu Beginn des Spiels die grundlegende Spielumgebung vorbereitet. Dies umfasst das Festlegen der Fenstergröße, das Aktivieren der 3D Darstellung sowie das Ausblenden des Mauszeigers, um eine klarere Benutzeroberfläche zu ermöglichen. Anschließend werden die Spieler eingerichtet, der Billardtisch initialisiert und alle relevanten Spielkomponenten wie Bälle und Queue hinzugefügt. Auch die Benutzeroberfläche wird hier vorbereitet, bevor das Spiel startet.

Die Methode `draw()` wird in jedem Frame aufgerufen und sorgt dafür, dass das Spielgeschehen kontinuierlich aktualisiert wird. Sie aktualisiert die Kameraperspektive, rendert die Spielszene und berechnet die Spiel Logik, einschließlich der Bewegungen und Kollisionen der Bälle, mit `updateGameLogic()`. Abschließend wird die Benutzeroberfläche dargestellt, um den aktuellen Spielstatus wie Punktestand und weitere wichtige Informationen anzuzeigen.

Zusätzlich übernimmt die Datei auch die Steuerung des Spiels. Über Mausklicks wird die Ausrichtung des Queues gesteuert und der Schuss ausgelöst, was eine direkte und intuitive Interaktion mit dem Spiel ermöglicht.

Die Datei Billard_ssds sorgt somit für die Interaktion aller Spielkomponenten und gewährleistet einen Spielfluss.

3.2.6 BillardCue

Die Klasse BillardCue verwaltet den Queue, der immer mit dem weißen Ball verbunden ist. Sie sorgt dafür, dass der Queue korrekt in Bezug auf den weißen Ball positioniert und ausgerichtet ist. Zu Beginn wird die Position des Queues relativ zum weißen Ball festgelegt. Der Queue wird als 2D Objekt in der Spielumgebung dargestellt, und es werden visuelle Anpassungen vorgenommen, um eine realistische Darstellung zu gewährleisten.

Es werden Methoden definiert, die es ermöglichen, die Schlagstärke durch Mausinteraktion zu steuern und den Queue nach jedem Schuss zurückzusetzen. Außerdem wird der Winkel des Queues aktualisiert, sodass er stets korrekt auf den Ball ausgerichtet bleibt, aber auch frei mit der Maus geändert werden kann.

3.2.7 GameController

Die Klasse GameController steuert den Spielfluss, indem sie den Spielerwechsel koordiniert und Ereignisse während des Spiels überwacht. Sie bestimmt den ersten Spieler und ordnet ihm die Ballart zu. Während des Spiels verwaltet sie die Züge, erkennt Fouls und initiiert den Spielerwechsel. Zudem stellt sie die korrekte Aktualisierung des Punktestands sicher.

3.2.8 GameState Enum

Die GameState Enum definiert verschiedene Spielzustände, die den Verlauf des Spiels steuern. Diese Zustände helfen dabei, die Aktionen des Spiels korrekt zu verwalten und auf bestimmte Ereignisse zu reagieren. Die wichtigsten Zustände sind:

- **BREAKSHOT**: Der Zustand, in dem das Spiel mit dem ersten Schlag startet.
- **PLACE_WHITE_BALL**: Ein Zustand, in dem der Spieler die weiße Kugel nach einem Foul auf dem Tisch platzieren muss.
- **READY**: Der Zustand, in dem der Spieler bereit ist, seinen nächsten Schlag auszuführen.
- **WAITING**: Ein Zustand, in dem das Spiel auf eine Entscheidung nach der Animation wartet.
- **FOUL**: Der Zustand, der ausgelöst wird, wenn ein Foul begangen wird, zum Beispiel das Versenken der weißen Kugel.
- **FINISHED**: Der Zustand, in dem das Spiel abgeschlossen ist und der Gewinner ermittelt wurde.

Diese Zustände helfen dabei, das Spiel in die jeweiligen Phasen zu unterteilen und die Logik entsprechend anzupassen.

Ein zusätzlicher Vorteil der GameState Enum ist die Verbesserung der Performance. Wenn das Spiel beispielsweise im FOUL Modus ist, muss nicht mehr auf Kollisionen geprüft werden, was die Berechnungen optimiert und Ressourcen spart.

3.2.9 Player

Die Klasse Player repräsentiert einen Spieler im Billardspiel und verwaltet alle relevanten Informationen sowie Methoden zur Steuerung des Spielverlaufs. Jeder Spieler hat einen Namen, eine Punktzahl, einen Balltyp und eine Liste der versenkten Bälle. Zudem wird festgehalten, ob der Spieler gerade am Zug ist.

Die Klasse ermöglicht es, die Punktzahl basierend auf versenkten Bällen zu erhöhen. Bei der Ballzuweisung wird zwischen soliden und gestreiften Bällen unterschieden. Bei der Schwarzen Kugel werden spezielle Regeln in der Methode `handleBlackBall()` geprüft, darunter, ob der Spieler die richtige Ballart noch im Spiel hat und ob er die schwarze Kugel korrekt versenkt hat.

Die Gewinnerlogik wird in der Methode `awardPoints()` behandelt, die den Spieler zum Gewinner erklärt, sobald er alle Bälle richtig versenkt hat. Nach Beendigung des Spiels wird der Zustand `winGame` ausgelöst, der den Spielbildschirm anzeigt und den Gewinner verkündet.

Zusätzlich verwaltet die Klasse den Spielerwechsel mit den Methoden `startTurn()` und `endTurn()`, die festlegen, wann ein Spieler an der Reihe ist und wann der nächste Spieler übernimmt. Der Balltyp des Spielers wird über die Methode `assignBallType()` zugewiesen, wobei der gegnerische Spieler automatisch den anderen Balltyp erhält.

Die Methode `getStatus()` gibt eine kurze Übersicht über den aktuellen Status des Spielers, einschließlich Punktestand und ob er am Zug ist.

3.2.10 Pockets

Die Klasse `Pockets` repräsentiert die Taschen des Billardtisches, in denen die Bälle versenkt werden. Insgesamt gibt es sechs Taschen: vier an den Ecken und jeweils eine an der mittleren Position der linken und rechten Seiten.

Im Konstruktor werden die Koordinaten der Taschen basierend auf den übergebenen Eingabewerten festgelegt. Zusätzliche Anpassungen an den Positionen der Ecken erfolgen, um die Taschen an den Billardtisch anzupassen. Die Koordinaten werden in einer Liste gespeichert, um eine effiziente Verwaltung und Verwendung zu ermöglichen.

Die Methode `draw()` rendert jede Tasche auf dem Bildschirm. Sie durchläuft alle gespeicherten Koordinaten und verwendet die `PShape` Klasse, um jede Tasche als sphärische Form mit einer Textur darzustellen. Die Taschen werden leicht verschoben, um ihre korrekte Position auf dem Tisch zu erhalten. Die Textur, die auf jede Tasche angewendet wird, ist in der Variablen `blackTexture` gespeichert und wird aus einer externen Datei geladen.

Die Taschen sind entscheidend für das Spiel, da sie die Orte darstellen, an denen die Bälle versenkt werden. Ihre Darstellung auf dem Bildschirm bietet dem Spieler eine visuelle Rückmeldung über den Zustand des Spiels.

3.2.11 ShootingBar

Die Klasse `ShootingBar` stellt die Anzeige und Steuerung der Schussstärke im Billardspiel dar. Sie visualisiert eine horizontale Leiste, die den aktuellen Wert der Schusskraft anzeigt und es dem Spieler ermöglicht, die Stärke seines Schusses dynamisch zu steuern. Diese Leiste spielt eine zentrale Rolle im Spiel, da sie den Spieler direkt in den Prozess der Schussvorbereitung einbindet.

In der Konstruktor Methode werden die grundlegenden Parameter wie die Position auf dem Bildschirm, die maximale Stärke des Schusses sowie die Abmessungen der Leiste festgelegt. Der `BillardCue`, also der Queue des Spielers, wird ebenfalls übergeben, um eine Verbindung zwischen der Leiste und der aktuellen Schussstärke herzustellen. Zu Beginn ist die Schussstärke auf Null gesetzt, was bedeutet, dass der Spieler mit keiner Stärke schießt, bis er seine Maus bewegt.

Die Methode `updateStrength()` aktualisiert die Schussstärke basierend auf dem Fortschritt der Queue Animation. Durch die Anwendung der mathematischen Funktion `sin` wird der Fortschritt der Leiste von -1 bis 1 gemappt, um so eine sanfte Änderung der Schussstärke zu ermöglichen. Dies gibt dem Spieler ein Gefühl der Kontrolle und Präzision, da er durch das Halten der Maustaste die Stärke des Schusses feinjustieren kann.

Durch die visuelle, farbliche und dynamische Darstellung der Schussstärke wird dem Spieler eine klare Rückmeldung über die Stärke seines Schusses gegeben, was zu einer interaktiveren und ansprechenderen Spielerfahrung führt.

3.2.12 Table

Die Klasse Table modelliert einen Billardtisch, einschließlich seiner Wände, der unteren, oberen, rechten und linken Kante. Sie berechnet die Positionen der Ecken und der Taschen basierend auf den übergebenen Koordinaten der Wände und fügt diese in einer Liste zusammen. Eine Instanz der Pockets Klasse wird verwendet, um die Taschen des Tisches zu verwalten. Die `draw()` Methode ist verantwortlich für das Rendern des gesamten Tisches. Sie besteht aus den Unterfunktionen `drawTableTop()`, `drawTableLegs()` und `drawBorders()`, die jeweils die Tischoberfläche, die Beine und die Randbereiche zeichnen. Die Geometrie wird unter Berücksichtigung eines festen Abstands zu den Grenzen des Tisches berechnet, wobei Texturen zur Veranschaulichung verwendet werden.

3.2.13 VectorDrawer

Die Klasse VectorDrawer visualisiert die Schussrichtung des Queues und gibt eine provisorische Ansicht des Schusses in Rot, die zeigt, wo der Ball nach der Kollision hinrollen wird. Die Methode `draw()` berechnet den Kollisionspfad eines Balls, indem sie die Richtung des Queue verfolgt und eine Linie in diese Richtung zieht. Sobald der Queue einen Ball trifft, wird der prognostizierte Pfad des getroffenen Balls in Rot dargestellt, um die Schussrichtung des Balls nach der Kollision anzuzeigen. Die Methode `drawPredictedPath()` berechnet die zukünftige Position des Balls nach der Kollision, basierend auf der Impulsübertragung und der Geschwindigkeit des Balls. Diese Darstellung hilft, die Ballbewegung visuell vorauszuberechnen und die Schussplanung zu unterstützen.

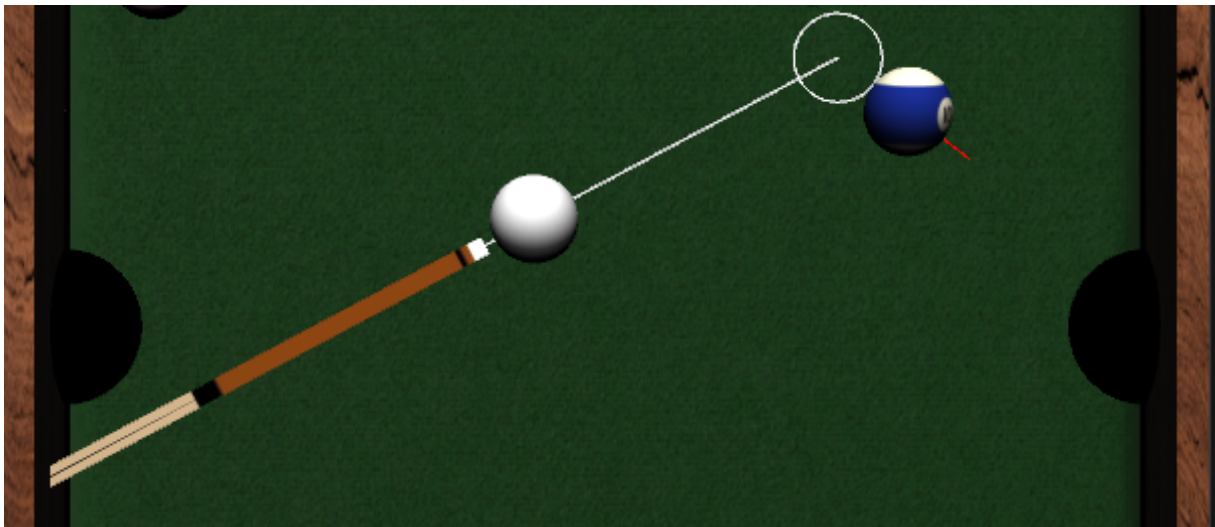


Abbildung 9: Schussrichtung

3.2.14 SoundController

Die Klasse SoundController spielt – entsprechend ihres Namens – die passenden Töne ab, sobald eine bestimmte Aktion ausgelöst wird. Zu diesen Aktionen gehören:

- Kollision zwischen zwei Kugeln,
- Kollision zwischen einer Kugel und der Bande,
- Kollision zwischen dem Queue und der weißen Kugel,
- Versenken einer Kugel.

Der Sound trägt maßgeblich zur Spielatmosphäre bei, indem die Interaktion nicht nur visuell, sondern auch akustisch unterstützt. Dadurch wird das Spielerlebnis immersiver und der Spielspaß erhöht.

4 Texturierung

Die Texturierung ist ein wesentlicher Bestandteil der grafischen Gestaltung, der die visuelle Qualität und Authentizität des Spiels erhöht. In unserer Implementierung werden verschiedene Texturen auf die Objekte angewendet, um deren Erscheinungsbild zu verfeinern und eine realistische Spielumgebung zu schaffen.

Texturen werden auf 3D-Modelle projiziert, um Oberflächen mit Details zu versehen. Für jedes Objekt wurde eine passende Textur ausgewählt, wobei auf eine korrekte Skalierung und Ausrichtung geachtet wurde, um Verzerrungen zu vermeiden. Die Texturen werden aus externen Bilddateien geladen und optimiert, um die Performance nicht zu beeinträchtigen.

Die folgenden Texturen wurden verwendet:

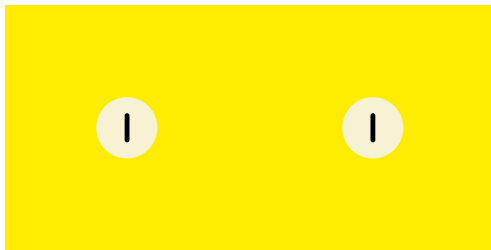


Abbildung 10: Ball Solid

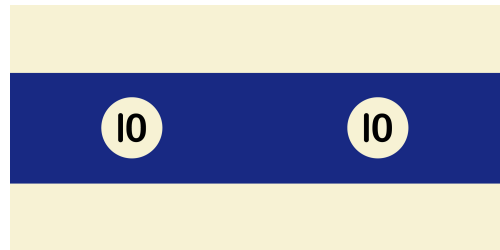


Abbildung 11: Ball Stripe

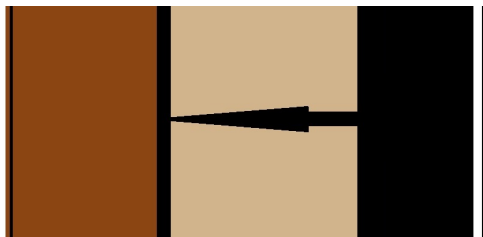


Abbildung 12: Queue



Abbildung 13: Holz

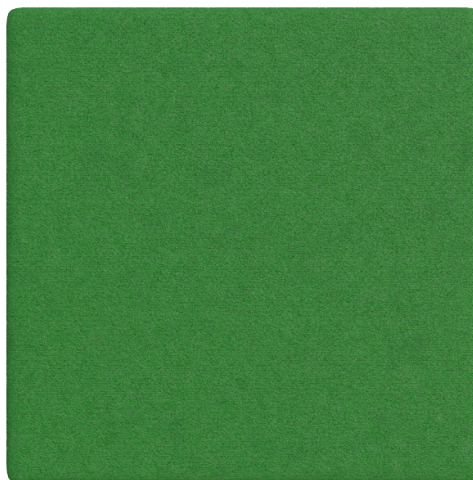


Abbildung 14: Green Pool

5 Artefakte

Wie in jeder Implementierung, ist es Artefakten nicht auszuweichen. Was wir tun können ist diese analysieren, Betrachten und somit verfeinern oder ganz entfernen.

Hier zeigen wir, was wir festgestellt haben, wie unser Programm funktioniert, was noch verbesserungspotenzial hat und wo noch lücken zu sehen sind.

5.1 3D-Effekt

Obwohl das Projekt ursprünglich in 3D konzipiert wurde und die Kameraperspektive den Tisch aus verschiedenen Winkeln betrachtet, ist der vollständige 3D-Effekt im aktuellen Spielstand noch nicht realisiert. Eine präzisere Modellierung ist erforderlich, um diesen Effekt zu erreichen. Dazu gehört unter anderem die exakte Darstellung der Tischkanten sowie die vollständige dreidimensionale Modellierung der Taschen, des Queues und eine passende Kameraperspektive.

5.2 Pockets

Die Modellierung des Billardtisches ist eine Herausforderung für sich. In diesem Projekt haben wir uns dazu entschieden, diesen Aspekt zu vereinfachen und die Taschen des Tisches als einfache Halbkreise darzustellen.

Diese Lösung ist nicht optimal, da die Taschen eines realen Billardtisches nicht nur in die Tischfläche integriert sind, sondern auch einen Teil der Kante einnehmen. Zudem besitzen die vier Ecktaschen jeweils eine spezielle Führung.

Dadurch entspricht die visuelle Darstellung des Billardtisches nicht vollständig der Realität. Dies kann auch dazu führen, dass gelegentlich zwei Bälle gleichzeitig in eine Tasche fallen.

5.3 Rotation

Die Rotation funktioniert korrekt, sofern die Textur des Bildes zentral zur Kamera projiziert wird. Dreht sich der Ball jedoch während des Spiels um, müssen Anpassungen vorgenommen werden. Eine Methode zur Lösung dieses Problems wurde bereits in einer vorherigen Sektion erläutert. Es gibt jedoch einen weiteren Sonderfall, in dem die Rotation fehlerhaft ist.

Dieses Problem tritt auf, wenn das obere oder untere Drittel der Textur zentral zur Kamera projiziert wird und der Ball entlang der X Achse geschossen wird. In diesem Fall erfolgt keine Rotation um die Y Achse.

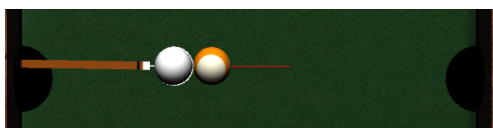


Abbildung 15: Vor dem Schuss

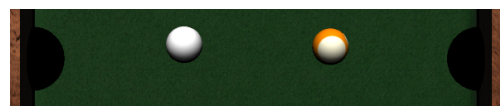


Abbildung 16: Nach dem Schuss

5.4 Vektoranzeige

Gelegentlich kann es vorkommen, dass beim äußerst knappen Kontakt der weißen Kugel mit einer anderen Kugel ein Vektor in Schussrichtung mit einem roten Pfeil visualisiert wird. Dieser kann irreführende Vorhersagen liefern, indem er den Eindruck erweckt, dass die Kugel getroffen wird, obwohl sie in Wirklichkeit knapp daran vorbeiläuft.

6 Vergleich mit anderen Arbeiten

6.1 Studienarbeit aus Koblenz

Ein Vergleichspunkt ist die Studienarbeit von Andreas Herschbach aus dem Jahr 2007. [3] Diese überzeugt sowohl durch eine detaillierte quantitative als auch qualitative Dokumentation, die die Spiel-Features umfassend beschreibt. Durch eine präzise Modellierung der Animationen in Maya 6.0 sowie die Erstellung von Texturen in Adobe Photoshop 7.0 und Wood Workshop konnte er, basierend auf seinen Screenshots, eine realistische 3D-Darstellung des Billardtisches, Queues und der Schatteneffekte umsetzen. Da uns lediglich die Dokumentation vorliegt, können wir das tatsächliche Spiel jedoch nicht direkt beurteilen.

Die wesentlichen Unterschiede zu unserem Projekt sind:

- **Modellierung und Texturierung:** Die Modelle in der Studienarbeit sind äußerst detailliert und realitätsnah gestaltet, was eine hohe visuelle Qualität ermöglicht.
- **Effet:** Die Spin-Mechanik wurde implementiert. Zitat von Andreas: „... wurde ein physikalisch nicht korrekter, aber spielerisch dennoch ansprechender Ansatz gewählt.“ [3]
- **Spielmodi:** Das Spiel bietet die Möglichkeit, zwischen den Varianten 8-Ball und 9-Ball zu wählen.

Zusammenfassend ist die Dokumentation dieser Arbeit sehr überzeugend. Es ist jedoch zu beachten, dass verschiedene Softwarelösungen verwendet wurden. Zudem wird nicht explizit erwähnt, in welcher Game-Engine das Projekt realisiert wurde. Aufgrund der verwendeten Tools ist es jedoch unwahrscheinlich, dass Processing zum Einsatz kam.

Im Gegensatz dazu basiert unser Projekt vollständig auf Processing und konzentriert sich auf eine reine Implementierung, ohne eine detaillierte Modellierung der Architektur vorzunehmen.

6.2 Online Billardspiele

Ein weiterer Vergleichspunkt sind verschiedene Online Implementierungen von Billardspielen im Browser. Als Beispiel betrachten wir das Spiel *The Best Russian Billiards* [4] aus dem Katalog der Online-Spieleplattform *CrazyGames*.

Im direkten Vergleich fallen einige deutliche Unterschiede auf, die in unserem Projekt nicht umgesetzt wurden:

- **Implementierung in Unity:** Das Spiel basiert auf der Unity-Engine, was eine flüssige Darstellung mit einer weichen und realistischen Oberfläche ermöglicht.

- **Atmosphärische Umgebung:** Durch verbesserte 3D-Grafik, passende Soundeffekte und Musik wird eine ansprechendere Spielatmosphäre geschaffen.
- **Online- und Multiplayer-Funktionalität:** Das Spiel bietet die Möglichkeit, direkt online gegen andere Spieler anzutreten.
- **Ranglistensystem:** Spieler werden mit ungefähr gleichstarken Gegnern zusammengebracht, was faire und spannende Matches ermöglicht.
- **In-Game-Shop:** Im Spiel gibt es einen Shop, in dem bessere Queues mit erspieltem Geld erworben werden können. Dies dient als Motivation, mehr Zeit im Spiel zu verbringen.

Natürlich erfordert eine solche Umsetzung erheblich mehr Entwicklungszeit und muss in diesem Fall durch Werbung auf der Webseite finanziert werden.

7 Fazit

In diesem Projekt haben wir erfolgreich ein Billardspiel simuliert, das grundlegende Physikberechnungen für die Kollisionen der Kugeln, die Rotation sowie das Einlochen durchführt. Wie bei jedem Programm gibt es auch hier Potenzial für Verbesserungen und Erweiterungen. In Zukunft könnten komplexere Spielmechanismen, wie zusätzliche Spielmodi oder eine Multiplayer Unterstützung für ein Online Spiel, implementiert werden. Unsere aktuelle Version bietet jedoch bereits viel Spaß, und durch die Implementierung haben wir wertvolle neue Erkenntnisse gewonnen.

Literatur

- [1] LEIFIphysik: März, 2025. Quelle: <https://www.leifiphysik.de/mechanik/impulserhaltung-und-stoesse/grundwissen/zentraler-elastischer-stoss>
- [2] Serlo: o.D. Quelle: <https://de.serlo.org/mathe/227927/gegenseitige-lage-von-zwei-kugeln>
- [3] Andreas Herschbach, Entwicklung eines virtuellen Billardspiels, Mai 2007. Quelle: <https://kola.opus.hbz-nrw.de/frontdoor/deliver/index/docId/128/file/Ausarbeitung.pdf>
- [4] Browser Billard Spiel, The Best Russian Billiards. Quelle: <https://de.crazygames.com/spiele/the-best-russian-billiards>